a Python SQL toolkit and Object Relational Mapper which provides the developers the full power and flexibility of SQL.

SQLAlchemy provides full support for enterprise-level design patterns and is designed for high-performing database access while maintaining efficiency and ease of use. We'll build a user authentication module, CRUD (Create, Read, Update, and Delete) REST APIs for object creation, retrieval, manipulation, and deletion. We'll also integrate a documentation utility called Swagger for creating API documentation, write unit and integration tests, learn application debugging, and, finally, check out different methods of deploying and monitoring our REST APIs on cloud platforms for production use.

For unit tests, we'll use pytest which is a full-featured Python testing tool—pytest is easy to write tests with and yet is scalable to support complex use cases. We'll also use Postman which is a complete REST API Platform—Postman provides integration tools for every stage of the API lifecycle, making API development easier and more reliable.

API deployment and monitoring are critical parts of REST API development; development paradigm changes drastically when it comes to scaling the APIs for production use cases, and for the sake of this book, we'll deploy our REST APIs using uWSGI and Nginx on a cloud Ubuntu server. We'll also deploy our REST APIs on Heroku which is a cloud platform that facilitates Flask app deployment and scaling out of the box.

Last but not least, we'll discuss debugging common Flask errors and warnings and debugging Nginx requests and check out Flask application monitoring ensuring least amount on the downtime for production use.

Introduction to RESTful Services

Representational State Transfer (REST) is a software architectural style for web services that provides a standard for data communication between different kinds of systems. Web services are open standard

web applications that interact with other applications with a motive of exchanging data making it an essential part of client server architecture in modern web and mobile applications. In simple terms, REST is a standard for exchanging data over the Web for the sake of interoperability between computer systems. Web services which conform to the REST architectural style are called RESTful web services which allow requesting systems to access and manipulate the data using a uniform and predefined set of stateless operations.

Since its inception in 2000 by Roy Feilding, RESTful architecture has grown a lot and has been implemented in millions of systems since then. REST has now become one of the most important technologies for web-based applications and is likely to grow even more with its integration in mobile and IoT-based applications as well. Every major development language has frameworks for building REST web services. REST principles are what makes it popular and heavily used. REST is stateless, making it straightforward for any kind of system to use and also making it possible for each request to be served by a different system.

REST enables us to distinguish between the client and the server, letting us implement the client and the server independently. The most important feature of REST is its statelessness, which simply means that neither the client nor the server has to know the state of each other to be able to communicate. In this way, both the client and the server can understand any message received without seeing the previous message. Since we are talking about RESTful web services, let's take a dive into web services and compare other web service standards.

Web services in a simple definition is a service offered by one electronic device to another, enabling the communication via the World Wide Web. In practice, web services provide resource-oriented, web-based interface to a database server and so on utilized by another web client. Web services provide a platform for different kinds of systems to communicate to each other, using a solution for programs to be able to communicate with each other in a language they understand (Figure 1-2).

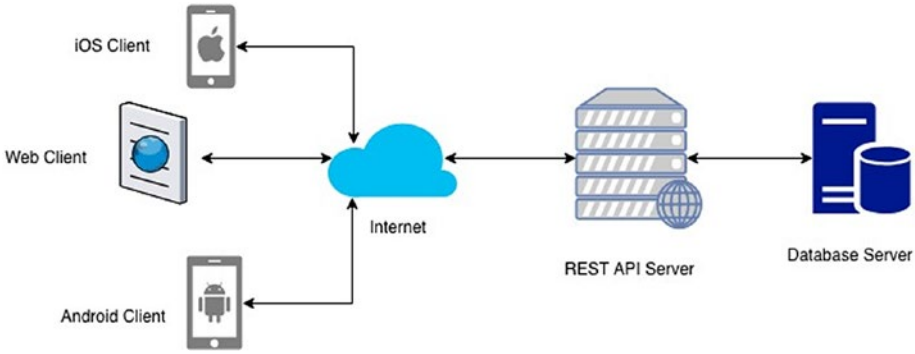

Figure 1-2. *REST architecture diagram*

SOAP (Simple Object Access Protocol) is another web service communication protocol which has been overtaken by REST in the recent years. REST services now dominate the industry representing more than 70% of public APIs according to Stormpath. They operate by exposing consistent interface to access named resources. SOAP, however, exposes components of application logic as services rather than data. SOAP is now a legacy protocol originally created by Microsoft and has a lot of other constraints when compared to REST. SOAP only exchanges data over XML, and REST provides the ability to exchange data over a variety of data formats. RESTful services are comparatively faster and less resource intensive. However, SOAP still has its own use cases in which it's a preferred protocol over REST.

SOAP is preferred when robust security is essential as it provides support for Web Services Security (WS-Security), which is a specification defining how security measures are implemented in web services to protect them from external attacks. Another advantage of SOAP over REST is its built-in retry logic to compensate for failed requests unlike REST in which the client has to handle failed requests by retrying. SOAP is highly extensible with other technologies and protocols like WS-Security, WS-addressing, WS-coordination, and so on which provides it an edge over other web service protocols.

Now, when we have briefly discussed web services—REST and SOAP—let's discuss features of REST protocol. In general, REST services are defined and implemented using the following features:

1. Uniform interface
2. Representations
3. Messages
4. Links between resources
5. Caching
6. Stateless

Uniform Interface

RESTful services should have a uniform interface to access resources, and as the name suggests, APIs' interface for the system should be uniform across the system. A logical URI system with uniform ways to fetch and manipulate data is what makes REST easy to work with. HTTP/1.1 provides a set of methods to work on noun-based resources; the methods are generally called verbs for this purpose.

In REST architecture, there is a concept of safe and idempotent methods. Safe methods are the ones that do not modify resources like a GET or a HEAD method. An idempotent method is a method which produces the same result no matter how many times it is executed. Table 1-1 provides a list of commonly used HTTP verbs in RESTful services.

Table 1-1. *Commonly used HTTP verbs useful in RESTful services*

Verb	CRUD	Operation	Safe	Idempotent
GET	Read	Fetch a single or multiple resource	Yes	Yes
POST	Created	Insert a new resource	No	No
PUT	Update/ Create	Insert a new resource or update existing	No	Yes
DELETE	Delete	Delete a single or multiple resource	No	Yes
OPTIONS	READ	List allowed operations on a resource	Yes	Yes
HEAD	READ	Return only response headers and no body	Yes	Yes
PATCH	Update/ Modify	Only update the provided changes to the resource	No	No

Representations

RESTful services focus on resources and providing access to the resources. A resource can be easily thought of as an object in OOP. The first thing to do while designing RESTful services is identifying different resources and determining the relation between them. A representation is a machine-readable explanation defining the current state of a resource.

Once the resources are identified, representations are the next course of action. REST provides us the ability to use any format for representing the resources in the system. Unlike SOAP which restricts us to use XML to represent the data, we can either use JSON or XML. Usually, JSON is the preferred method for representing the resources to be called by mobile or web clients, but XML can be used to represent more complex resources.

Here is a small example of representing resources in both formats.

Listing 1-2. XML Representation of a Book Resource

```
<?xml version="1.0" encoding="UTF-8"?>
<Book>
  <ID> 1 </ID>
  <Name> Building REST APIs with Flask </Name>
  <Author> Kunal Relan </Author>
  <Publisher > Apress </ Publisher >
</Book>
```

Listing 1-3. JSON Representation of a Book resource

```
{
      "ID": "1",
      "Name": "Building REST APIs wiith Flask",
      "Author": "Kunal Relan",
      "Publisher": "Apress"
}
```

In REST Systems, you can use either of the methods or both the methods depending on the requesting client to represent the data.

Messages

In REST architecture, which essentially established client–server style way of data communication, messages are an important key. The client and the server talk to each other via messages in which the client sends a message to the server which is often called as a request and the server sends a response. Apart from the actual data exchanged between the client and the server in the form of request and response body, there is some metadata exchanged by the client and the server both in the form of request and response headers. HTTP 1.1 defines request and response headers formats in the following way in order to achieve a uniform way of data communication across different kinds of systems (Figure 1-3).

Figure 1-3. *HTTP sample request*

In Figure 1-4, GET is the request method, "/comments" is the path in the server, "postId=1" is a request parameter, "HTTP/1.1" is the protocol version that the client is requesting, "jsonplaceholder.typicode.com" is the server host, and content type is a part of the request headers. All of these combined is what makes a HTTP request that the server understands.

In return, the HTTP server sends the response for the requested resources.

```
[
  {
    "postId": 1,
    "id": 1,
    "name": "id labore ex et quam laborum",
    "email": "Eliseo@gardner.biz",
    "body": "laudantium enim quasi est quidem magnam voluptate
            ipsam eos\ntempora quo necessitatibus\ndolor quam
            autem quasi\nreiciendis et nam sapiente accusantium"
  },
  {
    "postId": 1,
    "id": 2,
    "name": "quo vero reiciendis velit similique earum",
    "email": "Jayne_Kuhic@sydney.com",
```

```
  "body": "est natus enim nihil est dolore omnis voluptatem
          numquam\net omnis occaecati quod ullam at\nvoluptatem
          error expedita pariatur\nnihil sint nostrum voluptatem
          reiciendis et"
},
{
  "postId": 1,
  "id": 3,
  "name": "odio adipisci rerum aut animi",
  "email": "Nikita@garfield.biz",
  "body": "quia molestiae reprehenderit quasi aspernatur\naut
          expedita occaecati aliquam eveniet laudantium\nomnis
          quibusdam delectus saepe quia accusamus maiores nam
          est\ncum et ducimus et vero voluptates excepturi
          deleniti ratione"
},
{
  "postId": 1,
  "id": 4,
  "name": "alias odio sit",
  "email": "Lew@alysha.tv",
  "body": "non et atque\noccaecati deserunt quas accusantium
          unde odit nobis qui voluptatem\nquia voluptas
          consequuntur itaque dolor\net qui rerum deleniti ut
          occaecati"
},
{
  "postId": 1,
  "id": 5,
  "name": "vero eaque aliquid doloribus et culpa",
  "email": "Hayden@althea.biz",
```

```
    "body": "harum non quasi et ratione\ntempore iure ex
             voluptates in ratione\nharum architecto fugit
             inventore cupiditate\nvoluptates magni quo et"
  }]
```

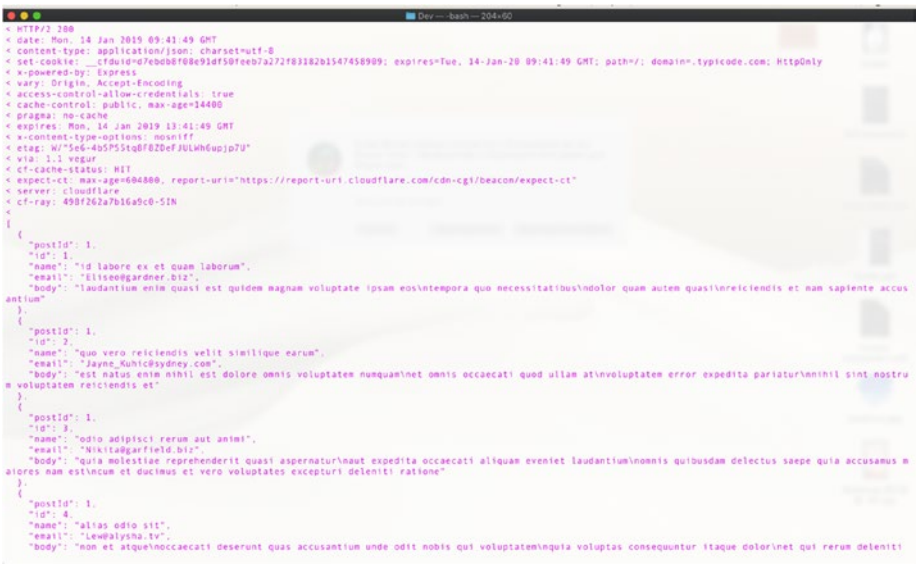

Figure 1-4. *HTTP sample response*

In the preceding figure, “HTTP/2” is the response HTTP version and “200” is the response code. The part below that till “cf-ray” is the response headers, and the array of post comments below “cf-ray” is the response body of the request.

Links Between Resources

A resource is the fundamental concept in the world of REST architecture. A resource is an object with a type, associated data, and relationships to other resources alongside a set of methods that can be executed on it. The resource in a REST API can contain link to other resources which should drive the process flow. Such as in the case of a HTML web page in

which the links in the homepage drive the user flow, resources in REST API should be able to drive the flow without the user knowing the process map.

Listing 1-4. A Book with Link to Buy

```
{
      "ID": "1",
      "Name": "Building REST APIs wiith Flask",
      "Author": "Kunal Relan",
      "Publisher": "Apress",
       "URI" : "https://apress.com/us/book/123456789"
}
```

Caching

Caching is a technique that stores a copy of a given resource and serves it back when requested, saving extra DB calls and processing time. It can be done at different levels like the client, the server, or a middleware proxy server. Caching is an important tool for increasing the API performance and scaling the application; however, if not managed properly, it results in the client being served old results. Caching in REST APIs is controlled using HTTP headers. Cache headers have been an essential part of HTTP header specifications and have been an important part of scaling web services with efficiency. In REST specification, when a safe method is used on a resource URL, usually the reverse proxy caches the results to use the cached data when the same resource is requested the next time.

Stateless

> Each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client
>
> —Roy Fielding

Statelessness here means that every HTTP response is a complete entity in itself and enough to serve the purpose of providing information to be executed without any need of another HTTP request. The point of statelessness is to defeat the purpose of accord with the server allowing intended flexibility in the infrastructure. To facilitate the same, REST servers provide enough information in the HTTP response that the client may need. Statelessness is an essential part of being able to scale the infrastructure enabling us to deploy multiple servers to serve millions of concurrent users given the fact that there is no server session state dependency. It also enables the caching feature of REST infrastructure as it lets the caching server to decide whether to cache the request or not, just by looking at the particular request irrespective of any previous requests.

Planning REST API

Here is a list of things we need to check while planning to create REST APIs:

1. Understanding the use case. It is really important to know why you are building the API and what services will the API provide.
2. Listing down API features to understand what all actions your APIs are going to do. This also includes listing down actions and grouping them together to tackle redundant endpoints.
3. Identify different platforms that'll use the API and provide support accordingly.
4. Plan long term on supporting growth and scaling the infrastructure.
5. Plan API versioning strategy ensuring continuous support is maintained over different versions of the APIs.

6. Plan API access strategy, that is, authentication, ACL, and throttling.
7. Plan API documentation and testing.
8. Understand how to use hypermedia with your APIs.

So, these are the eight important things to ensure while planning your API and are really crucial for developing a stable, production-focused API system.

API Design

Now let's look into API design. Here we'll cover the standards of designing REST APIs keeping in mind the list of things we just talked about.

Long-Term Implementation

Long-term implementation helps you analyze the flaws in design before actual implementation. This helps the developers to choose the right kind of platforms and tools to build upon making sure the same system can be scaled for more users later.

Spec-Driven Development

Spec-driven development enforces API design using definition and not just the code, which ensures that the changes are made to the codebase while the API design is intact. It is good practice to use a tool like API Designer to understand the API design before development which also lets you foresee the flaws. Tools like swagger or RAML let you keep the API design standardized and enable you to port the API to different platforms if needed.